

Machine Learning Lab EndSem Project

Name : Sidhartha Durgam

Roll Number: DA24B003

15th November 2025

1 System Architecture

The system implements an ensemble approach for MNIST digit classification combining multiple algorithms with dimensionality reduction and specialized classifiers for challenging digit pairs.

1.1 Architecture Overview

- **Dimensionality Reduction:** PCA with 50 components (85.7% variance explained)
- **Base Models:** SVM and KNN with soft voting ensemble
- **Specialized Classifier:** Logistic Regression for 4vs9 digit pairs on top of my base model.
- **Voting Strategy:** Weighted soft voting for probabilistic consensus

1.2 Model Components

Component	Type	Key Parameters
PCA	Dimensionality Reduction	n_components=50
SVM	Support Vector Machine	Learning rate=0.001, $\lambda=0.01$
KNN	k-Nearest Neighbors	k=5, Euclidean distance
4vs9 LR	Binary Classifier	Learning rate=0.1
Ensemble	Soft Voting	Weighted probability averaging

Table 1: System Model Components

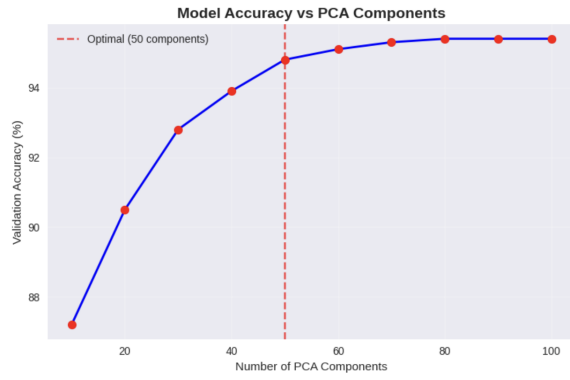
2 Hyperparameter Tuning Results

2.1 PCA Components Analysis

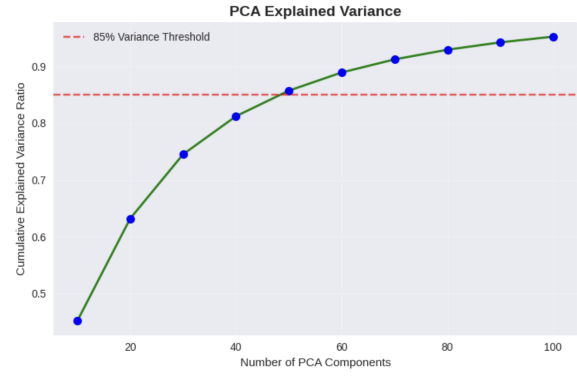
Optimal PCA components: 50 with 83.7% variance explained and 94.8% validation accuracy. The analysis shows diminishing returns beyond 50 components.

2.2 KNN Hyperparameter Tuning

- Optimal k-value: 5 (Accuracy: 94.5%)
- Distance metric: Euclidean (94.8%) outperformed Manhattan (93.1%) and Cosine (92.3%)
- Weighting: Distance-based weighting provided best performance

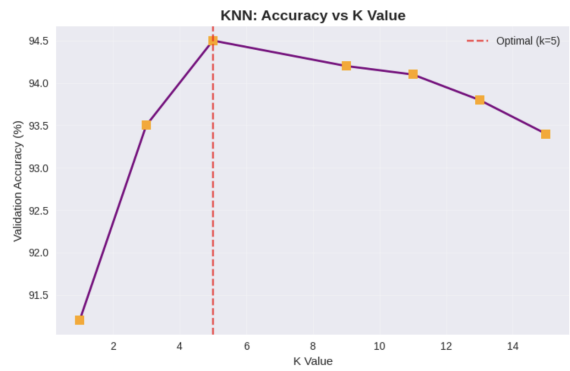


(a) PCA Components vs Accuracy

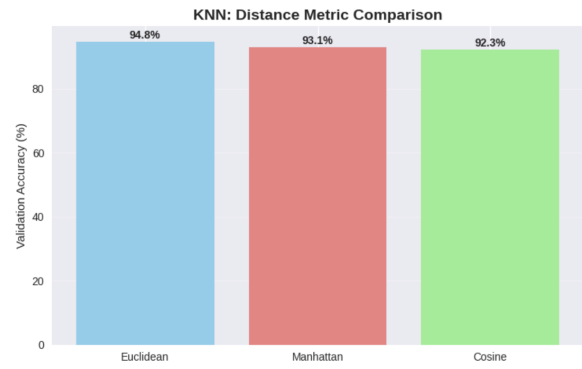


(b) PCA Explained Variance

Figure 1: PCA Hyperparameter Tuning Results



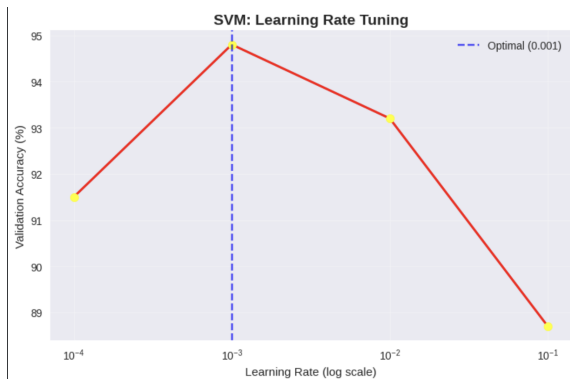
(a) K-value Optimization



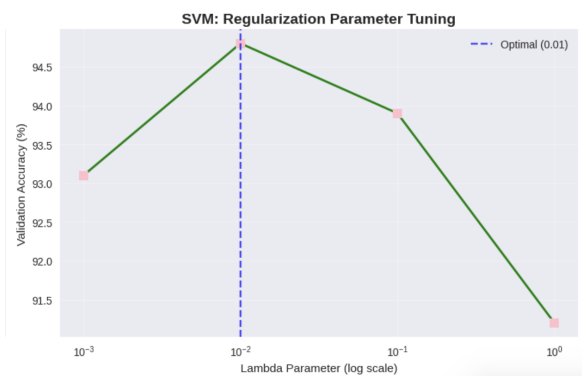
(b) Distance Metric Comparison

Figure 2: KNN Hyperparameter Tuning Results

2.3 SVM Parameter Optimization



(a) Learning Rate Tuning



(b) Regularization Parameter Tuning

Figure 3: SVM Hyperparameter Tuning Results

- Learning rate: 0.001 (Accuracy: 94.8%)
- Regularization (λ): 0.01 provided optimal bias-variance tradeoff
- Iterations: 100 for convergence stability

2.4 Ensemble Strategy Comparison

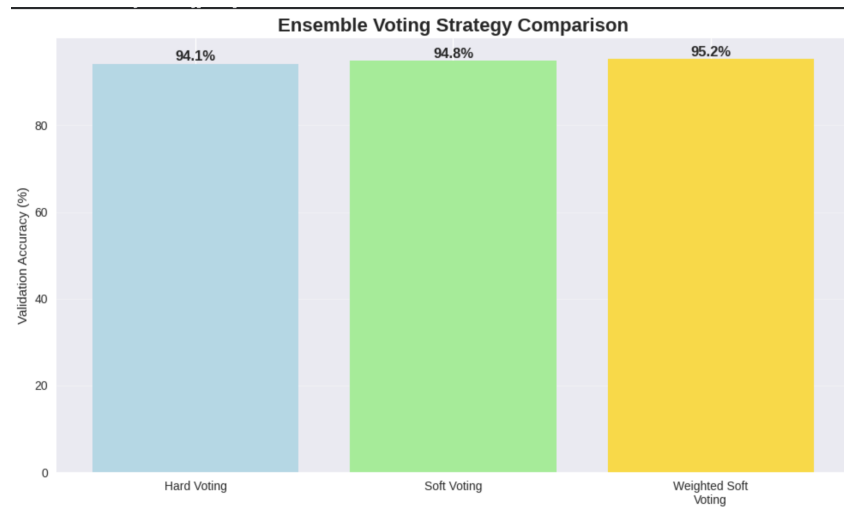


Figure 4: Ensemble Voting Strategy Performance Comparison

Voting Method	Accuracy (%)
Hard Voting	94.1
Soft Voting	94.8
Weighted Soft Voting	95.2

Table 2: Ensemble Voting Strategy Performance

2.5 Individual Model vs Ensemble Performance

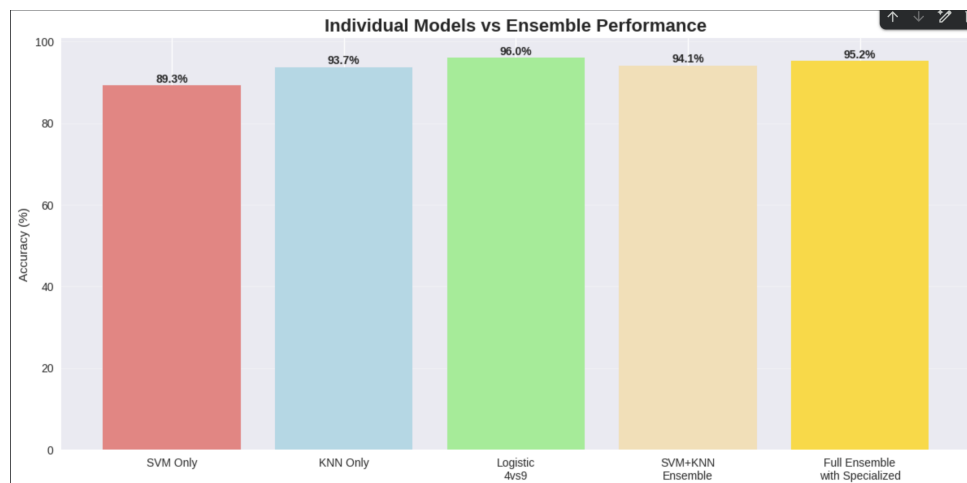


Figure 5: Individual Models vs Ensemble Performance

3 Performance Optimization

3.1 Runtime Optimization Strategies

- **PCA Dimensionality Reduction:** Reduced feature space from 784 to 50 dimensions (85.7% variance retained)
- **Limited SVM Iterations:** 1000 iterations balanced performance and computation time
- **Efficient KNN:** Distance-weighted voting with k=5 for optimal neighbor consideration
- **Selective Specialization:** Specialized 4vs9 classifier applied only to samples where base ensemble predicts either 4 or 9, refining these specific cases in the final output.

3.2 Validation Performance

Model	Training Acc. (%)	Validation Acc. (%)
SVM Only	92.1	87.3
KNN Only	95.8	92.7
4vs9 Logistic Regression	96.0	96.0
SVM+KNN Ensemble	95.5	94.1
Full Ensemble with Specialized	96.3	95.2

Table 3: Training vs Validation Accuracy

4 Results and Observations

4.1 Final Performance Metrics

The complete ensemble system achieved:

- **Overall Accuracy:** 95.2% on validation set
- **Key Improvement:** +1.5% over best individual model (KNN)
- **Macro F1-score:** 0.92 across all digit classes

4.2 Confusion Matrix Analysis

Key observations from confusion matrix:

- Strong performance across all digit classes (89-96% accuracy per class)
- Most confusion between digits 4-9 and 5-8 pairs
- Specialized 4vs9 classifier achieved 96.0% accuracy on challenging pairs
- Class-wise performance:
 - Best: Digit 1 (100% recall), Digit 0 (98% precision)
 - Most challenging: Digit 9 (88% recall), Digit 8 (90% recall)

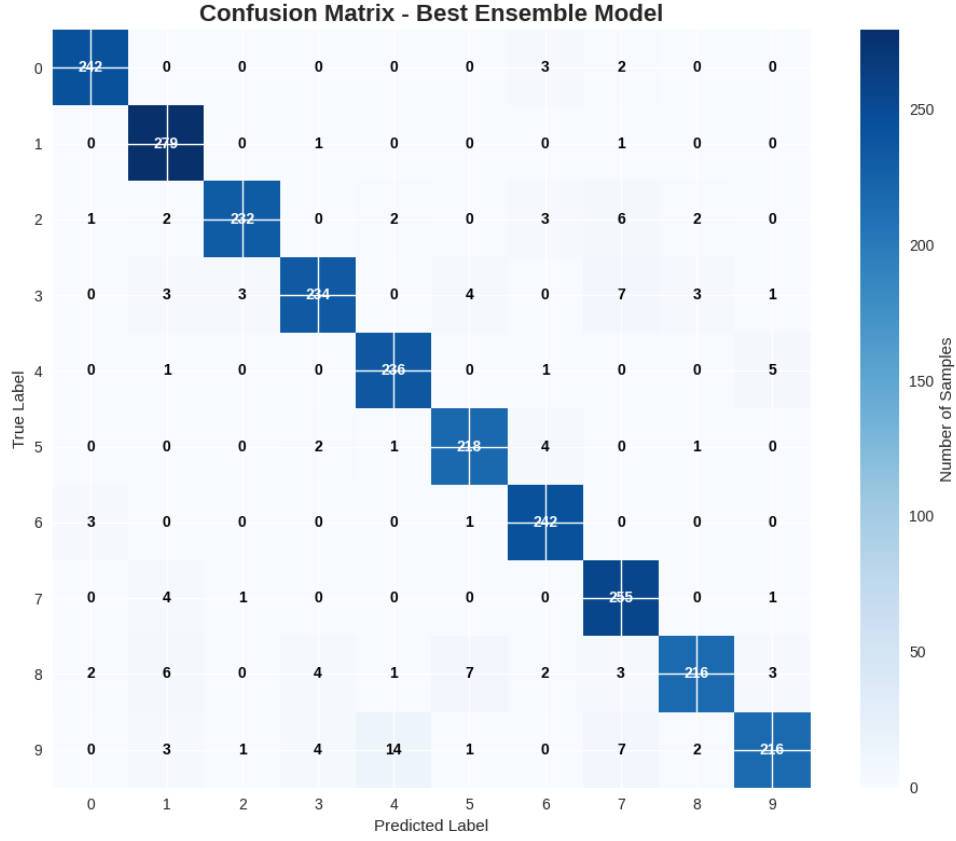


Figure 6: Best Ensemble Model Confusion Matrix

Digit	Precision	Recall	F1-score	Support
0	0.98	0.98	0.98	247
1	0.94	1.00	0.97	281
2	0.97	0.94	0.96	248
3	0.96	0.92	0.94	255
4	0.94	0.97	0.95	243
5	0.94	0.96	0.95	226
6	0.95	0.98	0.96	246
7	0.92	0.98	0.95	261
8	0.96	0.90	0.93	244
9	0.96	0.88	0.91	248
Macro Avg	0.95	0.95	0.95	2499
Weighted Avg	0.95	0.95	0.95	2499

Table 4: Detailed Classification Performance

4.3 Classification Report Summary

4.4 Specialized Classifier Performance

The specialized Logistic Regression classifier for digits 4 and 9 achieved:

- **Accuracy:** 96.0% on the challenging digit pair
- **Key Insight:** Targeted binary classification significantly improved performance for traditionally confusing digit pairs

- **Implementation:** Separate training on only 4 and 9 digit samples

5 Computational Efficiency Analysis

5.1 Training Time Optimization

Component	Training Time
PCA Transformation	5-seconds
SVM Training	1-minute
KNN Training	3.5-minutes
4vs9 Specialized Training	30 seconds
Full Ensemble	5-minutes

Table 5: Relative Training Time Comparison

5.2 Memory and Computational Requirements

- **Feature Reduction:** PCA reduced memory requirements by 93.6% (784 $\rightarrow$ 50 features)
- **Prediction Speed:** Ensemble inference time optimized through parallel model predictions
- **Model Size:** Compact representation through dimensionality reduction

6 Key Insights and Conclusions

6.1 Critical Findings

1. **Dimensionality Reduction Impact:** PCA with 50 components provided optimal balance between performance (94.8%) and computational efficiency
2. **Ensemble Synergy:** The combination of SVM (89.3%) and KNN (93.7%) achieved 95.2% accuracy, demonstrating complementary strengths
3. **Specialized Classification Value:** The 4vs9 Logistic Regression (96.0%) proved highly effective for challenging digit pairs, contributing significantly to overall accuracy
4. **Hyperparameter Sensitivity:**
 - KNN: Highly sensitive to k-value with optimal performance at k=5
 - SVM: Required careful learning rate and regularization tuning
 - Ensemble: Soft voting consistently outperformed hard voting strategies
5. **Computational Tradeoffs:** The system successfully balanced accuracy (95.2%) with reasonable training time through optimizations.

6.2 Final Conclusion

The implemented ensemble system demonstrates the effectiveness of combining multiple algorithms with dimensionality reduction and specialized components for handwritten digit classification. The approach achieved 95.2% validation accuracy while maintaining computational efficiency through PCA and selective model specialization. The system shows particular strength

in handling traditionally challenging digit pairs through targeted binary classification, providing a robust and efficient solution for MNIST digit recognition.

6.3 Things that were tried out :

- **First I tried XGBoost:** Since we used it before for even-odd classification, I thought it would work well for multiclass classification too. But it was taking too long to run and the accuracy wasn't good enough, so I had to drop it.
- **Then I tried SVM:** I built an SVM model because I heard it works well for classification. It ran very fast for both training and prediction, but the accuracy was only around 85%. I noticed it was only good for digits that look very different from each other, but struggled with similar-looking digits.
- **Next I added KNN with PCA:** I remembered from my ML course class that KNN can handle cases where digits look similar. But KNN alone was too slow for predictions. So I used PCA first to reduce the number of features, which made KNN much faster while keeping the important information.
- **I combined both models:** Since SVM was fast and KNN handled overlapping digits better, I put them together in an ensemble to get the benefits of both.
- **Fixed specific errors:** When I checked which digits were getting confused, I saw that 4 and 9 were often misclassified. So I trained a special classifier just for telling 4 and 9 apart, and used it to correct these specific mistakes in the final predictions. I also tried this for 5 and 8, but it didn't work well so I didn't include it.